DHBW Stuttgart

Karsten Keßler

Datenbanksysteme
9. Teil – SQL DML

Kurs Informatik 24ITA

DHBW Stuttgart

Übersicht der Gesamtvorlesung

Thema	Inhalte (Auszug)
1 Einführung	Einführung, Begriffe, Eigenschaften Funktionen, Datenintegration
2 Datenbankmodelle	Architektur und SPARC, Verschiedene Datenbankmodelle, Entwicklung
3 ERM	Entity-Relationship-Modell, konzeptueller Entwurf, Entwurfsprozess
4 Relationales Modell	Relationales Datenbankmodell im Detail
5 Relationale Algebra	Operatoren, Anwendung, Nützlichkeit
6 Normalformen	Normalisierung von Datenbanken, funktionale Abhängigkeiten, Synthesealgorithmus
7 Vektordatenbanken	Einstieg in Vektordatenbanken
8 SQL DDL	Datendefinition mit SQL
9 SQL DML	Datenmanipulation mit SQL
10 Transaktionen	ACID, Grundlegende Schwierigkeiten bei Mehrbenutzersystemen, gängige Lösungen
11 Integrität	Integrität im relationalen Modell wiederholt, Umsetzung (in SQL)
12 Sichten	Sichten (in SQL), Vor- und Nachteile, Umgang mit Sichten
13 Indizes	Interne Strukturen, Nützlichkeit für Anfragen
14 Optimierung	Optimierung von Datenbanken
15 SQL Server	Administration des SQL Server
16 Handout	Handout: Datenbanksysteme – Zusammenfassung

SQL als Data Manipulation Language (DML)

```
SELECT  Vorname, Nachname
FROM    Personen
WHERE   PLZ = '39106'
        OR  PLZ > '70000'
```

- Attribute (Projektion)
- Relationenname
- Selektionsbedingung
- Kombinationen durch Boole'sche Operatoren möglich

SQL arbeitet auf Multimengen

→ d.h. die in der Relationenalgebra übliche Duplikatunterdrückung muss erzwungen werden

```
SELECT  DISTINCT Nachname
FROM    Personen
WHERE   PLZ > '70000'
```


Verknüpfungen in SQL

```
SELECT  *
FROM    Personen, Kursteilnahme
WHERE   Personen.PNR = Kursteilnahme.PNR
```

- SQL nutzt das Kreuzprodukt als Verknüpfungsgrundlage
- JOIN (Verbund) über Verbundbedingung herstellen

Wie stellt man eine Verknüpfung einer Relation mit sich selbst her?

```
SELECT  t_eins.PNR, t_zwei.PNR
FROM    Personen t_eins, Personen t_zwei
WHERE   t_eins.Telefon = t_zwei.Telefon
```

Erzeugt eine Tabelle von Personenpaaren mit gleicher Telefonnummer

Vereinigungen

Verknüpfen von Relationen durch **Vereinigung**

```
SELECT  A, B, C
FROM    R1
UNION
SELECT  A, B, C
FROM    R1
```

Verknüpfen von Relationen durch **Differenzbildung**

```
SELECT  *
FROM    Personen
WHERE   Personen.PNR NOT IN
                { SELECT PNR
                  FROM   Pers_Telefon }
```


Überblick: SQL DML

SFW - „SELECT FROM WHERE“

SELECT: ... gibt Projektionsliste an
... nutzt arithmetische Operationen und Aggregatfunktionen

FROM: ... legt verwendete Relationen fest
... führt Umbenennungen durch
... Verknüpfung durch karthesisches Produkt

WHERE: ... legt Selektionsbedingung fest
... legt Verbundbedingung fest (u.a. Join statt Kreuzprodukt)
... erlaubt Verschachtelung (weiterer SFW-Block)

GROUP BY: ... ermöglicht Gruppierungen innerhalb einer Relation

HAVING: ... stellt Selektionsbedingung an diese Gruppen auf

SQL SFW: **S**ELECT **F**ROM **W**HERE

Näheres zur „**FROM**“-Klausel

```
SELECT *
FROM Buch, Autor
```
→ Erzeugt karthesisches Produkt !

```
SELECT *
FROM Verein v1, Verein v2
```
→ Verknüpfung einer Relation mit sich selbst

```
SELECT *
FROM Buch JOIN Ausleih
        ON Buch.InventarNr = Ausleih.InventarNr
```
→ Join statt Kreuzprodukt (SQL-92)

```
SELECT *
FROM Buch JOIN Ausleih USING (InventarNr)
```

```
SELECT *
FROM Buch NATURAL JOIN Ausleih
```

```
SELECT Ergebnis.InventarNr
FROM (Buch NATURAL JOIN Ausleih) AS Ergebnis
```


Innerer und Äußerer Verbund

A	B
1	2
2	3

Links

B	C
3	4
4	5

Rechts

```
SELECT *
FROM Links NATURAL JOIN Rechts
```

A	B	C
2	3	4

NATURAL JOIN

```
SELECT *
FROM Links OUTER JOIN Rechts
```

A	B	C
1	2	?
2	3	4
?	4	5

OUTER JOIN

```
SELECT *
FROM Links LEFT OUTER JOIN Rechts
```

A	B	C
1	2	?
2	3	4

LEFT OUTER JOIN

```
SELECT *
FROM Links RIGHT OUTER JOIN Rechts
```

A	B	C
2	3	4
?	4	5

RIGHT OUTER JOIN

SELECT

SELECT [DISTINCT] *{attribut | arithmetischer Ausdruck | aggregat-funktion}*
FROM ...

```
SELECT Buch.Name, Autor.Name
FROM Buch, Autor
WHERE Buch.ANR = Autor.ANR
```

```
SELECT B.Name, A.Name
FROM Buch B, Autor A
WHERE Buch.ANR = Autor.ANR
```


WHERE

SELECT ... FROM ...
WHERE bedingung

A) Konstanten-Selektion	Name = ‘karl‘
B) Attribut-Selektion	Preis < Budget
C) Verbundbedingung	Buch.Autor = Person.Name
D) Bereichsselektion	Note BETWEEN 1.0 AND 4.0
E) Ungewissheitsselektion	Name LIKE ‘%mueller‘
F) Null-Selektion	Autor IS NULL
G) Quantifizierte Bedingungen	ALL, ANY, SOME, EXISTS
H) Logische Verknüpfungen aller Bedingungsarten	AND, OR, NOT

Verschachtelung von Anfragen

```
SELECT ... FROM ...
WHERE Name IN {SFW-Block}
```

```
SELECT Nachname
FROM Personen
WHERE 1.0 IN ( SELECT Note FROM Prüfung WHERE Pnr = Personen.PNR)
```

```
SELECT ISBN
FROM Buch_Exemplare
WHERE EXISTS (SELECT * FROM Ausleihe WHERE INR=Buch_Exemplare.INR)
```


Aggregatfunktionen und Weitere Operatoren

Aggregatfunktionen: COUNT, SUM, AVG, MAX, MIN

```
SELECT count(*) FROM Buch_Versionen
```

```
SELECT avg(Note) FROM Prüfung WHERE Fach = 'Datenbanken 1'
```

ACHTUNG!

```
SELECT avg(ALL Note) FROM Prüfung WHERE Fach = 'Datenbanken 1'
```

```
SELECT MatNr FROM Prüfung
WHERE Note < ( SELECT avg(ALL Note) FROM Prüfung WHERE Fach = 'Datenbanken 1')
```

Weitere Operationen

```
SELECT 2 FROM Ergebnis
```

Zugriff auf zweite Spalte

```
SELECT ISBN, Preis * 0,88 AS DollarPreis FROM Buch
```

Gruppierungen, Quantoren, Mengenoperationen

GROUP BY und HAVING

```
SELECT COUNT(*), PersNr
FROM    Ausleihe
GROUP BY PersNr
HAVING count(*) > 1
```

Berechnet pro Person die Anzahl der von ihr ausgeliehenen Bücher, sofern diese Zahl größer als 1 ist.

Quantoren und Mengenoperationen

```
SELECT PersNr, Immatrikulationsdatum
FROM Studenten
WHERE MatNr = ANY ( SELECT MatNr FROM Prüfung)
```

```
SELECT Note
FROM Prüfung
WHERE MatNr = 'S-001205'
AND Note ⍰ ALL (SELECT Note FROM Prüfung WHERE MatNr = 'S-001205')
```

Schlechteste Note eines Studenten

Sortieren und Dreiwertige Logik mit SQL

```
SELECT MatNr, Note
FROM Prüfung
WHERE Fach = 'Datenbanken 1'
ORDER BY Note ASC
```

Dreiwertige Logik in SQL

true, known, false
AND, OR

Änderungen in der Datenbank

UPDATE <relation>
SET <attribut 1> = <ausdruck 1>
...
<attribut n> = <ausdruck n>
WHERE ...

DELETE

FROM <Relation>
WHERE ...

INSERT
INTO <relation> (attrib 1, ..., attrib n)
VALUES (konst 1, ..., konst n)

Vielen Dank für Ihre Aufmerksamkeit!